

Model-Driven Software Development

Introduction



*Revolutionizing software development
using domain-specific languages,
models, and automated code
generation*

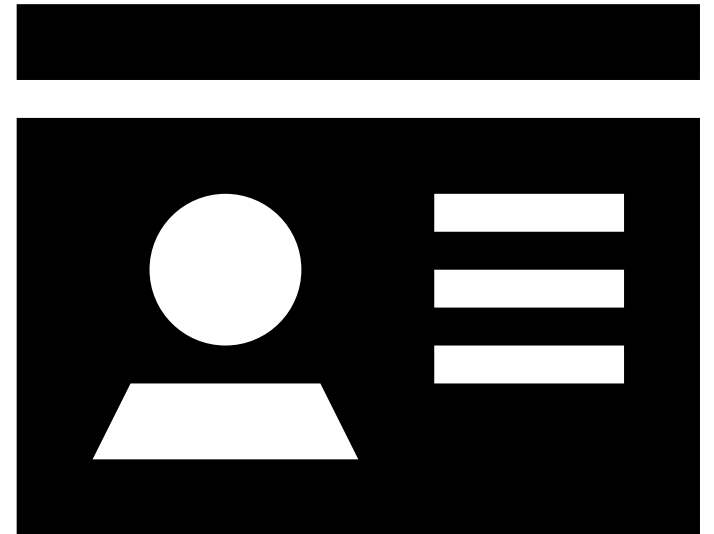
You first

You first

Your name

Your education

Favorite programming language



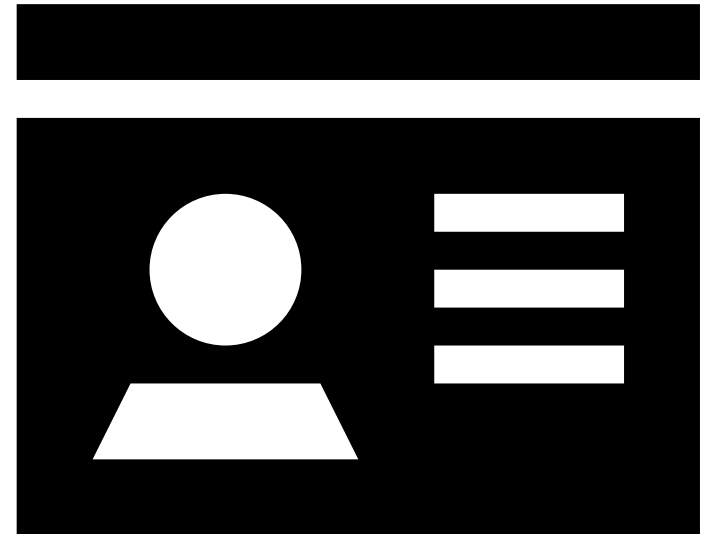
You first

What do you expect from this course?

What do you want to learn?

How is the video related to this course?

<https://pollev.com/mica>



Now me

Now us

Miguel Campusano: Software engineer for robotic applications

Robots: flying (drones), grounded, mobile, reactive systems

Software: architecture, program comprehension, test, live programming

Combination of software and robots

Education: Computer Science from University of Chile

Nationality: Chile (sorry for the accent!)

Programming Language: Xtext* (Pharo Smalltalk)

Now us

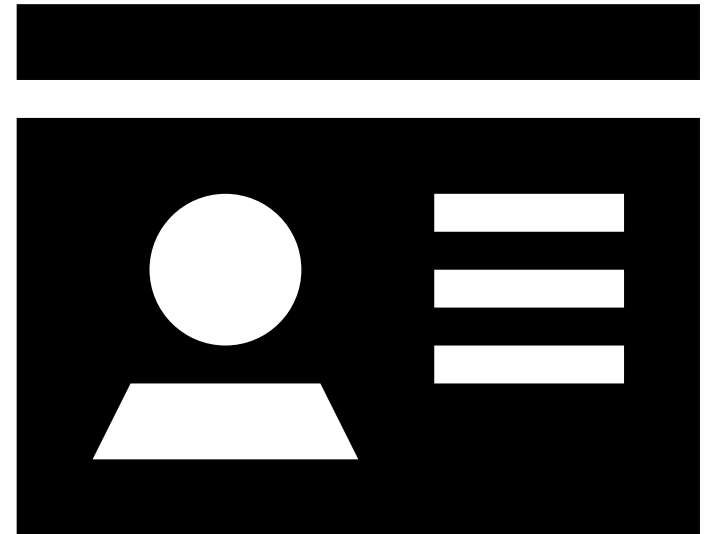
Contact

Miguel Campusano

mica@mmmi.sdu.dk

itslearning

Discord?



Course itself

Course itself

Solving *domain specific problems*: Model-Driven Software Development

Use of models, modeling of models (metamodels)

Design and describe metamodels

Via DSLs

Functional DSLs

Interpreter, code generation

Coding is more than *writing code*

Modeling and Programming Experience

Course itself

Web

itslearning

Discord

Lectures

Monday 8.30 – 11.30 (lectures)

Wednesday 8.15 – 11.30 (exercises)

Plan: mix of theory & practice (develop a DSL)

MDSD Project

Wednesday supervision

Monday supervision: on demand

Online

Course itself

Special thanks to Ulrik Pagh Schultz Lundquist
Lecture of this course for 10 years, until I took over
Now: Head of Center – SDU UAS Center

If this course works for you, you can thank him too!
If not ...



MDSD

Language, models and problems

SDU

Miguel Campusano

SDU Software Engineering

Today

Motivation

Bugs

Complexity vs Productivity

Here Be Dragons

Language, models, and problems

Course overview / MDSD Project

Motivation

Bugs

Motivation 1: Bugs

As soon as we started programming, we found to our surprise that it wasn't as easy to get programs right as we had thought.

Debugging had to be discovered. I can remember the exact instant when I realized that a large part of my life from then on was going to be spent in finding mistakes in my own programs

- Maurice Wikles, 1949

Motivation 1: Bugs: Ariane 5, flight 501

Software reused between Ariane 4 and 5
(good!)

Different flight characteristics caused
arithmetic overflow (not good)

Appropriate software handler disabled
for efficiency, causing complete failure in
controller (really bad)

Results: severe financial loss



Motivation 1: Bugs: Therac-25

Radiation therapy with two different modes: low and high current

Combination of causes

- Race condition

- Lack of hardware interlock

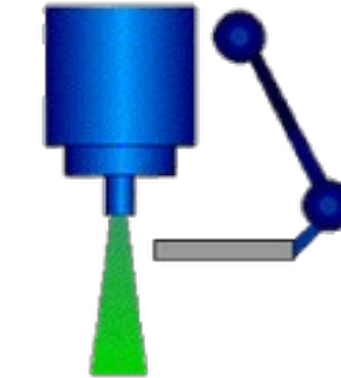
- Improper error reporting

- Fast sequence of improbable keystroke

- ...

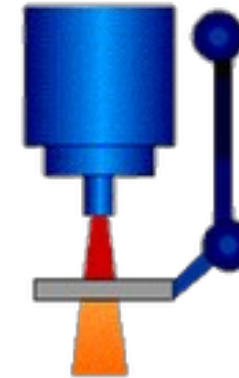
Results: loss of human life

low current
electron beam
was scanned
across the field



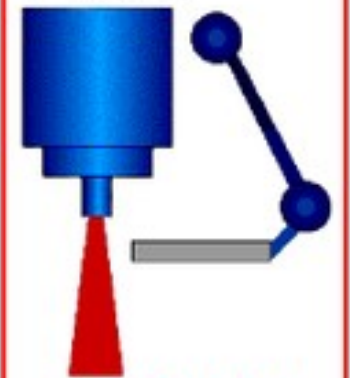
Electron Mode

high current
electron beam
was tracked
at the target



X-Ray Mode

high current
electron beam
with no target
> 'lightning'



THE PROBLEM

Motivation 1: Bugs: more?

Any more examples?

<https://pollev.com/mica>

Motivation 1: Bugs

Programmers cannot be expected to produce bug-free code

Software is an incredibly complex artifact

Representation gap: cognitive distance between real-world versus code

Some systems are safety-critical

Humans might be harmed / risk of loss of human life

Privacy violations, breakdown of critical infrastructure

Severe or critical loss of money for the company

System physically damaged due to software bugs

How to deal with this combination?

Motivation 1: Bugs: Solution

Automated code generation: correct by construction

- Develop correct code generator

- Automatically generate applications

- Profit! (well, not really)

Verification and Validation

- Manually implement software and formal verification of requirements

Testing and more ...

All angles adopted for safety-critical systems

Motivation 1: Bugs: Solution

Automated code generation: correct by construction

- Develop correct code generator

- Automatically generate applications

- Profit! (well, not really)

Verification and Validation

- Manually implement software and formal verification of requirements

- Testing and more ...

Hypothesis: should be much more widely adopted!

Clarification

Code generation vs model-based execution

Automated code generation



Technologies

Web tooling (Vue.js, Angular), modeling tools, DSLs

Compilers, generators, ...

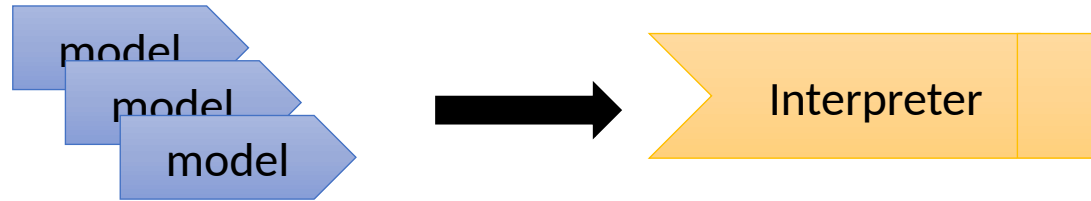
Alternatives

OO frameworks

Standard wizards found in IDEs

Outsourcing

Model-based execution



Technologies

JavaScript frameworks/API (jQuery), script engines, modeling tools, DSLs
Interpreters, virtual machines, ...

Alternatives

OO frameworks

Model-based tools that already *exist* (more or less)

Outsourcing

Clarification

Code generation vs model-based execution

Automated code generation: Swagger.IO

The screenshot displays the Swagger.IO web editor interface. The top bar shows the project name 'banking-dotnet_...' and the version 'v1-oas3'. The left sidebar contains navigation tabs: Info, Tags, Servers, and Schemas. The main editor area shows the OpenAPI specification in YAML format, with line numbers 1 through 36. The specification includes an API title 'Banking dotnet Core Web API', a description 'A simple api to use banking services', and a list of endpoints under the 'Bank' tag. The endpoints are: POST /api/Bank/create (Create a new user), POST /api/Bank/login (Login to the system with the given user), POST /api/Bank/create-account (Create an account for the given user), and POST /api/Bank/withdraw (Withdraw money from the given account of the given customer). The right sidebar shows the 'Banking dotnet Core' API title and a list of generated code snippets for various languages: aspnetcore, go-server, inflector, jaxrs-jersey, jaxrs-resteasy, jaxrs-resteasy-eap, nodejs-server, python-flask, scala-akka-http-server, and spring. The 'Server Stub' option is selected.

```
1 openapi: 3.0.0
2 info:
3   version: "v1-oas3"
4   title: Banking dotnet Core Web API
5   description: A simple api to use banking services
6   termsOfService: None
7 paths:
8   /api/Bank/create:
9     post:
10      tags:
11        - Bank
12      summary: Create a new user
13      description: "This is an example of how the BODY
14                  should be structured\n\t\t\"Nicolai
15                  Oksen\""
16      operationId: ApiBankCreatePost
17      responses:
18        '200':
19          description: OK
20          content:
21            text/plain:
22              schema:
23                $ref: '#/components/schemas/
24                  /CustomerDTO'
25            application/json:
26              schema:
27                $ref: '#/components/schemas/
28                  /CustomerDTO'
29            text/json:
30              schema:
31                $ref: '#/components/schemas/
32                  /CustomerDTO'
33      deprecated: false
34      requestBody:
35        content:
36          application/json:
37            schema:
38              type: string
39          text/json:
40            schema:
41              type: string
```

Banking dotnet Core

Client SDK

Server Stub

Documentation

Download API

Bank

POST /api/Bank/create Create a new user

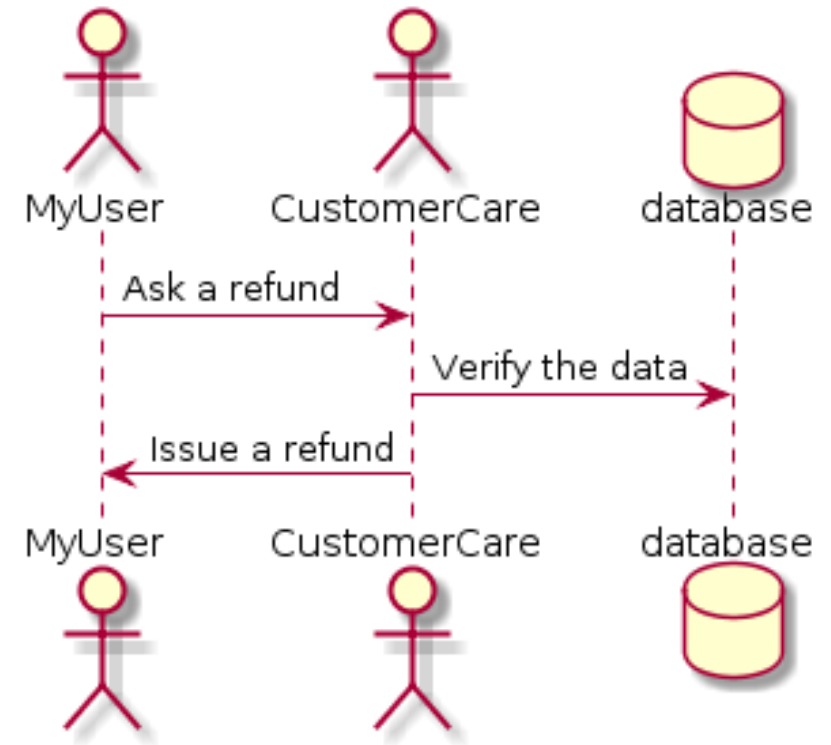
POST /api/Bank/login Login to the system with the given user

POST /api/Bank/create-account Create an account for the given user

POST /api/Bank/withdraw Withdraw money from the given account of the given customer

Automated code generation: PlantUML

```
@startuml
actor MyUser
actor CustomerCare
database database
MyUser -> CustomerCare : Ask a refund
CustomerCare -> database : Verify the data
CustomerCare -> MyUser : Issue a refund
@enduml
```



<https://tomassetti.me/domain-specific-languages/>

Automated code generation: Airbus

Airbus extensively uses automated code generation

Increasing degree of autogenerated code

Modeling/Programming using SCADE (LUSTRE)

Certified code generator (trains, cars, nuclear plants)

	A310	A320	A340
Digital Units	77	102	115
Size in M	4	10	20
Errors per 100K	hundreds	dozens	<10

A340 Auto-generated code

Fly-by-wire: 70%

Automatic flight control: 70%

Display computer: 50%

Warning and maintenance: 40%

Model-based execution: Web development

```
JS Save Fork Settings  
  
$('ul').children().slice(1,-2).css('background-color','salmon');  
$('ul').children().first().has('b').css('background-color','violet');  
$('ul').children().last().has('em').css('background-color','lime');  
  
$( "ul" ).click(function( event ) {  
    var target = $( event.target );  
  
    if ( target.is( "b" ) ) {  
        target.css( "background-color", "red" );  
    }  
});
```



Model-based execution: Robotics@Odense

The image displays two overlapping windows from the Universal Robots Graphical Programming Environment (UR GPE).

The background window is the UR GPE interface, titled "Universal Robots Graphical Programming Environment". It shows a program named "seek_with_force" with a tree view on the left containing "BeforeStart", "Robot Program", and "Thread_1". The "Robot Program" section is expanded, showing a sequence of steps: "Wait: 0.01", "MoveL", "Seek w force", "If force()<50", "Stop_force_seek", and "return_5mm". The "If" step is selected, and its configuration is shown in the main area. The "If" configuration includes the expression "force()<50" and a checked option "Check expression continuously". Below the configuration are buttons for "Add Elself", "Remove Elself", and "Add Else". The bottom of the window features a "Simulation" button, a "Real Robot" button, and a "Speed" slider set to 100%.

The foreground window is a mission editor, titled "Missions" and featuring the "MiR MOBILE INDUSTRIAL ROBOTS" logo. It shows a list of missions on the left, including "Go to charging station", "Adjust Localization", "Docking", "Move", "Move to coordinate", "Relative Move", "Set Targetpoint", and "Switch Map". The "Move" mission is selected, and its details are shown in the main area. The details include a "Groups" tool bar at the top with icons for "Move", "Battery", "Logic", "Error Handling", "Sound/Light", "PLC", and "Email". Below the tool bar, the mission steps are listed: "Move to A", "Move to B", "Go to charging station", and "Move to C". The "Actions" section below the mission list states: "Actions used in missions are all found in the Groups tool bar at the top of thee window." (Note the typo "thee"). The "Move" section shows a "Move" button.

Motivation

Complexity vs Productivity

Motivation 2: Complexity vs Productivity

- Software complexity is (always) increasing

 - Complexity is a significant risk to completion and correctness

 - Managed through abstractions, ultimately as models

- Models vs Code

 - Large mental gap

 - Models = documentation, often not maintained

- Repetitive programming

 - Recurring code patterns that cannot be abstracted

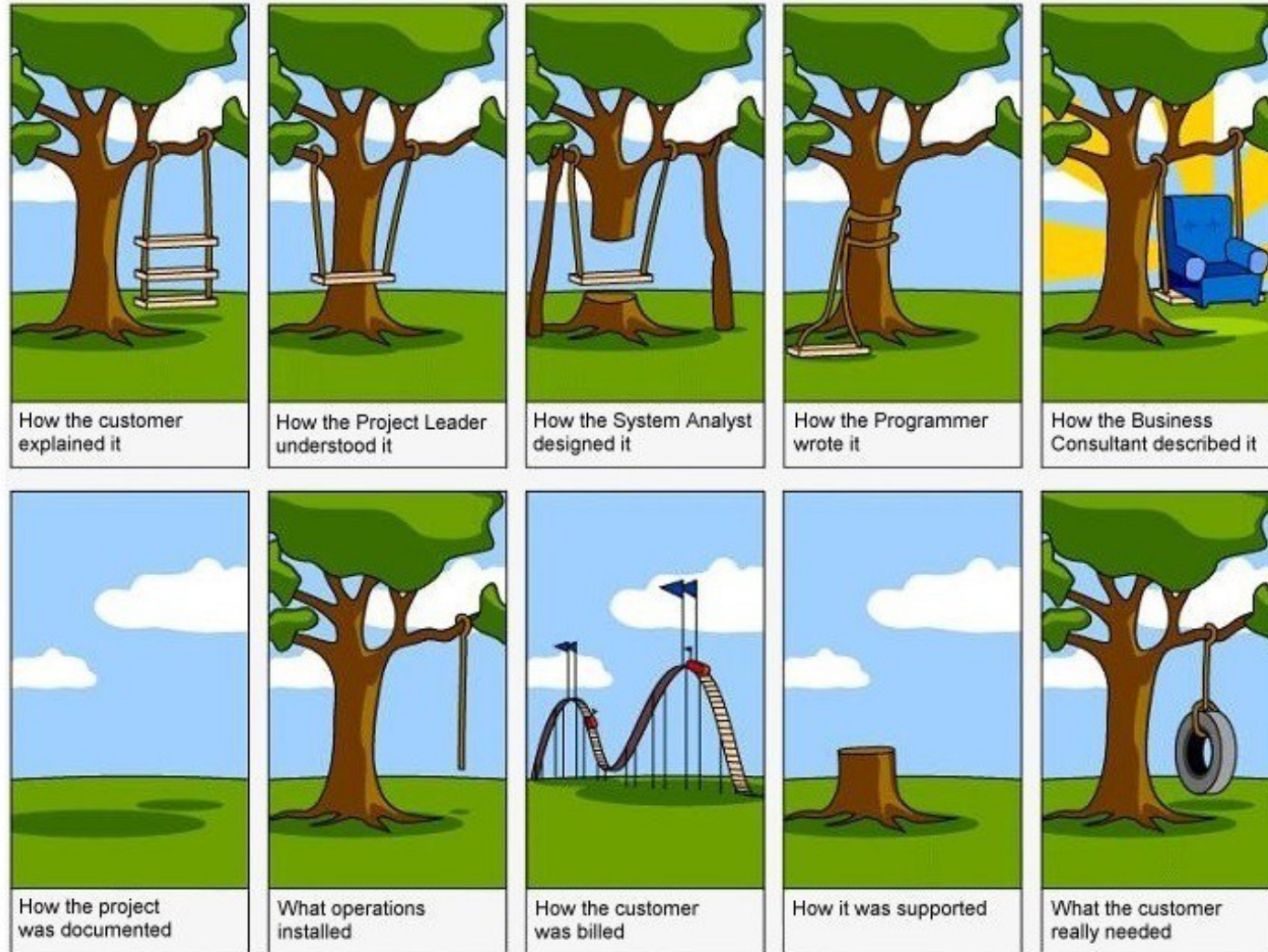
- Critical threats

Motivation 2: requirements, models & code



<https://www.youtube.com/shorts/23oXoBXxbyA>

Motivation 2: requirements, models & code



Motivation 2: CvsP: Solution

MDSD: Model-Driven Software Development

Formal model

UML, simulink, DSLs

Syntax and semantics

High-level and domain-specific

Relies on domain-specific notation

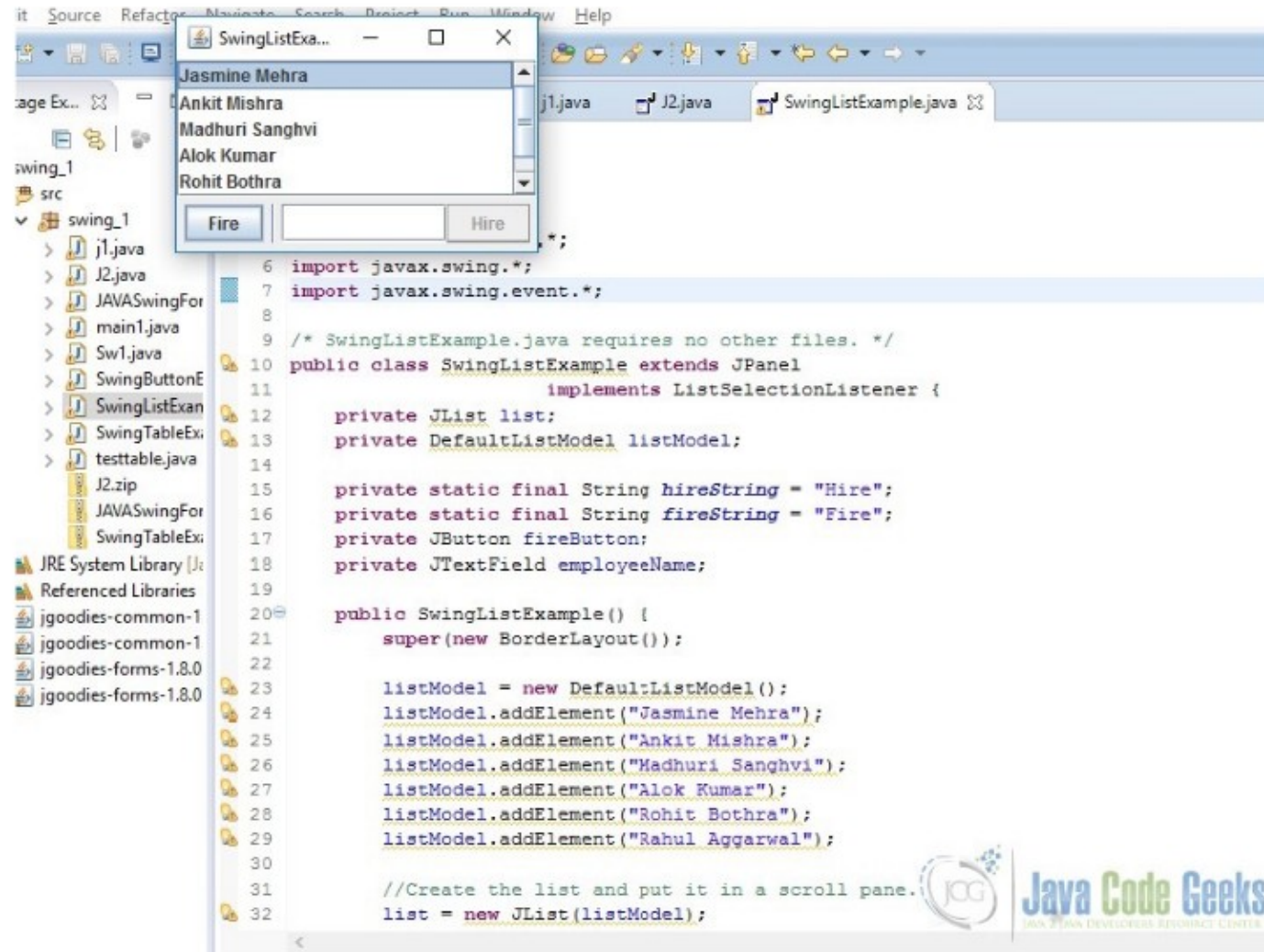
Code generation or model interpretation

Potentially huge gains in productivity (e.g., describe repetitive code once)

Verify safety, timing, security, ... , properties

Requires good tool support!

Motivation 2: CvsP: GUI Programming

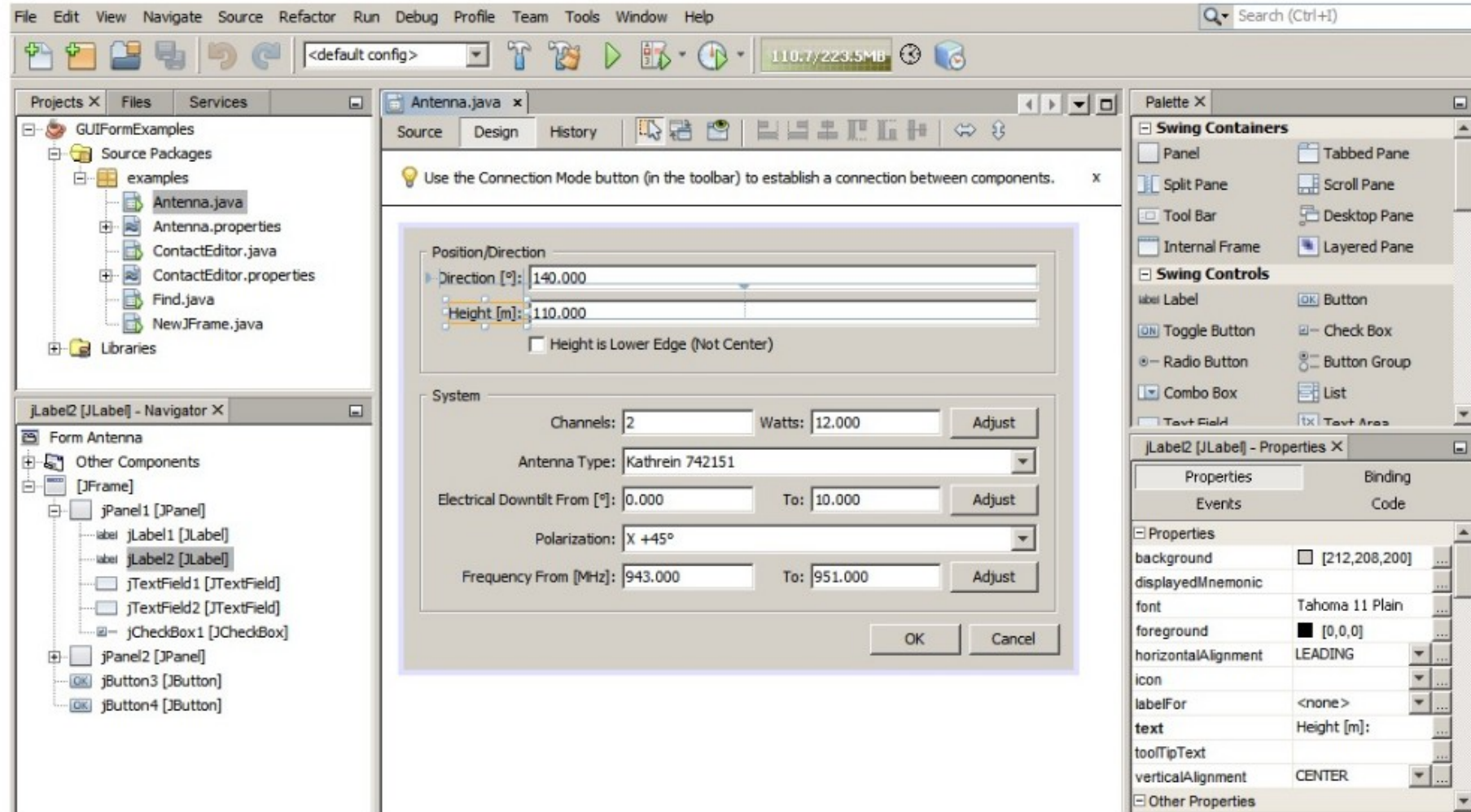


Motivation 2: CvsP: GUI Programming

```
</div>
<div class="menu-item">
  
</div>
</header>
<section class="container" id="home">
  <div class="col col1">
    <div class="intro1 animate__animated animate__flipInX"> <h1>Hello, I am Bibek Ghosh</h1>
    <div class="intro2"><h2>An aspiring Software developer</h2></div>
    <p class="intro">I enjoy challenges and finding a solution to a problem always fascinates
    <p class="intro3">I WRITE CODE</p>
  </div>
  <div class="col col2">
    
  </div>
</section>
<section class="projects" id="projects">
  <div class="project-section-title">
    <h2>Projects section </h2>
  </div>
  <div class="project-container">
    <div class="project-item">
      <div>
```

Source: <https://www.pexels.com/photo/programming-data-on-a-computer-14553707/>

Motivation 2: CvsP: GUI Generation



Motivation 2: CvsP: Cyber-physical systems

- Wind turbines highly dependent on control

 - Safety in violent wind conditions

 - Efficient operation in standard wind conditions

- Models play a central role (e.g., simulink)

 - Efficient, quick-to-use, domain-specific notation

 - Analysis and testing

- PLC controller code written manually from model

 - Programming errors

 - Mismatch when model evolves

Motivation 2: CvsP: PLC-Link

PLC-Link

Automatic translation of simulink and statecharts to PLC code

Gamesa G10X-4.5MW

40% of code generated from models

Saved about half a year of development time



Motivation

Here be dragons

Motivation 3: Here be dragons



Motivation 3: Understanding Prog Lang

What is a Prog Lang? How do we define/implement them?

Compilers, interpreters, virtual machines are everywhere, how do they work?

What is a Turing machine, what is Turing completeness?

What are Regular Expressions (RE)? relation between regular expressions and state machines? Context-Free Grammars (CFG)? RE vs CFG?

How do type systems relate to verification and validation? What does soundness and completeness mean for type systems?

Visual programming languages? Low-code development?

...

AI: Do we need to do/learn this?

AI can replace all of this?

Before: AI for drones and self-driving cars

Present: Prompt-engineering

Future: Humans?

<https://pollev.com/mica>

Languages, models and problems

Programming languages

Aspects investigated in this course

- General-purpose vs domain-specific

- Declarative vs imperative/OO/ ...

- Code generation vs interpretation

- MDSD as a mean to reduce evolution impact

Approach

- Making declarative, domain-specific languages

- Using interpretation and code generation

Domain-specific programming languages

Definition (almost)

- Computer programming language (BNF, ...)*

- Language nature (GCC linker scripts, ...)*

- Limited expressiveness (COBOL, ...)*

- Domain focus (always domain-specific)

Categories

- External DSLs: SQL, XML, CSS...

- Internal DSLs: hidden in fluent APIs (jQuery, LINQ, ...)

DSL Advantages

Improving development productivity

Communication with domain experts

Change in execution context

Alternative computation model

Opportunities for verification, ...

Order of magnitude gain when done right!

DSL Disadvantages

Language cacophony

Cost of building

Higher complexity

Ghetto language

Blocking abstraction

Languages and models

DSL

Textual (graphical)

Constructs, grammar, keywords from abstraction over domain

Interpret or generate code

Defined using semantic model or metamodel

Model

Graphical (textual)

Elements, metamodel, notation from abstraction over domain

Interpret or generate code

Defined using semantic model or metamodel

Domain-specific

Key Idea: Abstractions specific to domain

Problem domain / Family of problems

What is a relevant domain?

Interesting problems to solve

Solving problems is difficult (expressiveness, correctness)

Sensible abstractions (cohesion)

...

Course itself 2

Course itself 2

Solving *domain specific problems*: Model-Driven Software Development

Use of models, modeling of models (metamodels)

Design and describe metamodels

Via DSLs

Functional DSLs

Interpreter, code generation

Coding is more than *writing code*

Modeling and Programming Experience

Course itself 2

Metamodeling: meta, languages ideas, internal DSLs

External DSLs in Xtext: Syntax, Grammar

Advance Xtext: Parsing, Scope & Validation, DSL Design

Advance Topics: Grammar to metamodel, Prog Languages, Prog eXperience

Course itself 2: exam

Written exam (3 hours)

Part 1: 30 min – MCQ with multiple answers

Part 2: 1.5 hours – Open-ended questions

Course itself 2: exam

Written exam – Part 1: MCQ with multiple answers

NO AIDS

Focus on course content: lectures and slides

Problem Solving

e.g., according to certain properties, you should choose the proper model

Course itself 2: exam

Written exam – Part 2: Open-ended questions

Aids: everything but no online and no AI

Questions based on semester work: Project

Design and implement a DSL

Open problem: you choose the problem to be solved using DSLs

Size: any size, but recommended 2-3

There will be an individual component on the project

At the end, you will *extend* your group project with a DSL topic

Course itself 2: project

Group project: *Anything* related to the course

Key ideas: MDSD + another course you are interested

We (with TA) can define idea together

I will provide some ideas later in the week (before Thursday)

E.g., IoT configuration, Multi-robot planning, Videogames, BDD, etc.

You can invite external people: use of domain experts!

Course itself 2: project

Group project: course topics

- Problem description and solution design

- Metamodeling

- Internal DSL implementation → a way to easy use your metamodel

- External DSL implementation → the same, but harder

 - Grammar definition → grammar DSL and practical problems

 - Code generation → from your DSL, you generate Prog Lang valid code

- Advanced DSL topics → individual extension

Course itself 2: exercises

Use exercises to work in group project

We will post guidelines per each working week

TA is 100% available during exercise hours

You will post your progress in itslearning, where you receive feedback

Public post, so anyone can get inspiration (or post feedback)

NO AI: we want to review your progress, not a *solution* that is not yours

Course itself 2: Exercise, Projects, and Exam

Everything is connected

You will work on your project, every week, to study and prepare for the exam

However, with more knowledge, questions get harder

And, early answers will be modified too

i.e., if week 1 is DSL design, by week 10 your design may (it will!) change

So? If exam asks about DSL design, week 1 design is a (very) weak answer

Be aware: when preparing for the exam, do not assume an early semester exercise answer is a 12-grade type answer

Summary

(Domain-specific) *programming* Language

So, what is a programming language?

DSLs → domain-specific (programming?) language

Is ___ a Prog Lang?

Python / Java / C#

C / C++

Assembly

HTML / CSS / JSON / YAML / XML

Spreadsheets / Scratch / ...

(Domain-specific) *programming* Language

So, what is a programming language?

DSLs → domain-specific (programming?) language

Is ___ a Prog Lang?

Python / Java / C#

C / C++

Assembly

HTML / CSS / JSON / YAML / XML

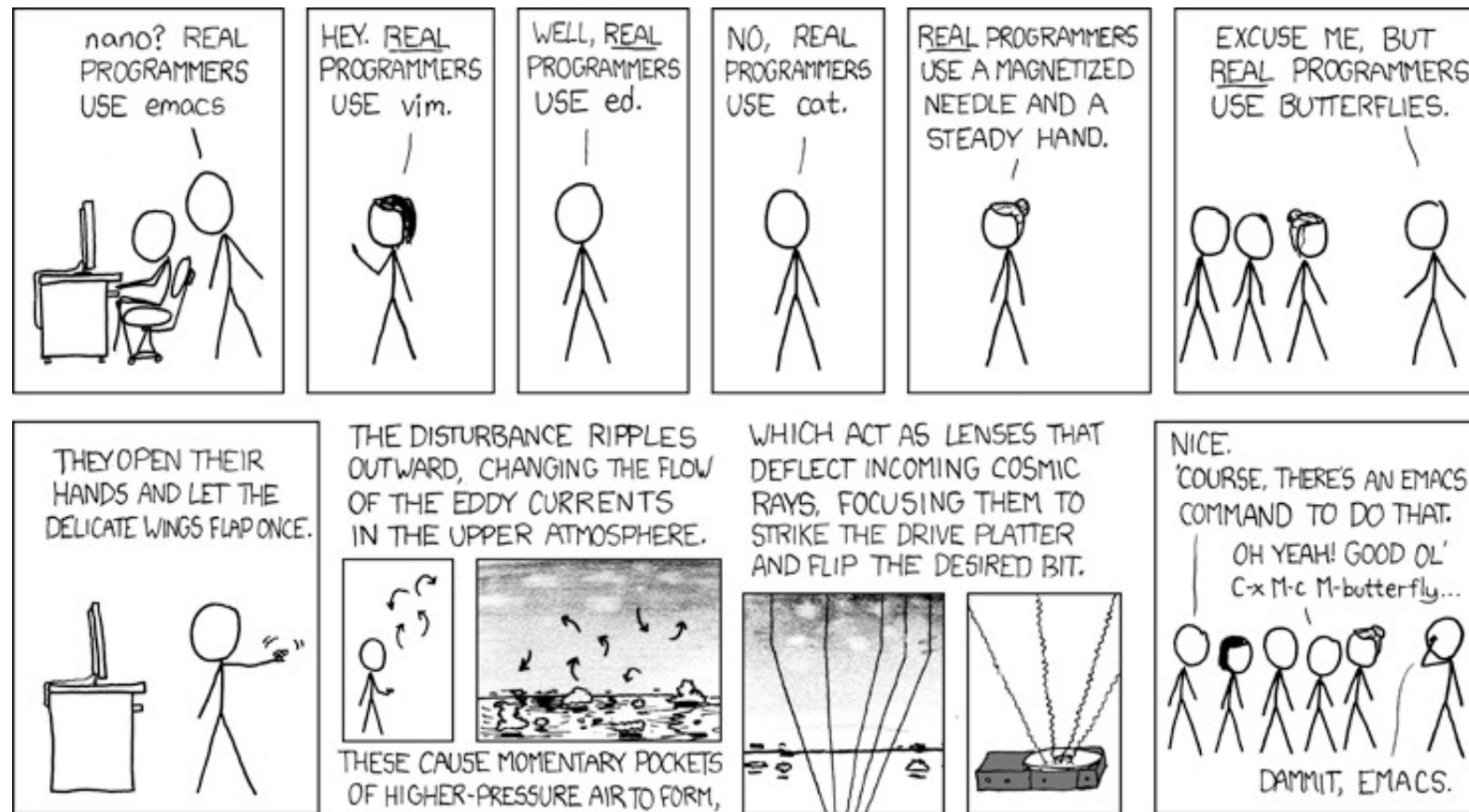
SpreadSheets / Scratch / ...



(Domain-specific) *programming* Language

So, what is a programming language?

DSLs → domain-specific (programming?) language



Summary

Motivation

(Domain-Specific) Programming Languages (?)

Course Introduction



Before next class

Post a DSL example on itslearning

